



Subject Name-WEB DESIGNING

Course Code:-BECS25-103

UNIT-C

Cascading Style Sheets



UNIT- C TOPICS- 2

- **Working with block elements and objects;**
- **Working with Lists and Tables**
- **CSS Id and Class;**
- **Box Model**
 - **Introduction**
 - **Border properties**
 - **Padding Properties**
 - **Margin properties**



Working with block elements and objects

Working with block elements and objects in CSS primarily involves understanding their default behavior and how to manipulate it using the display property and the CSS Box Model.

- Every HTML element has a default display value, depending on what type of element it is.



Working with block elements and objects

HTML block elements are used to create the logical and semantic layout of a web page. They help to organize the content into meaningful sections and make it easier for browsers, search engines, and site visitors to understand the structure and meaning of different parts of the web page. **Inline elements** are used to make useful block elements, like adding anchor links.

There are various tags that you can use to create blocks, such as `<div>`, `<p>`, `<table>`, and so on.

All the HTML elements can be categorized into two categories:

- **Block-level Elements**
- **Inline Elements**



Working with block elements and objects

Block-level Elements

- A block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element.
- A block-level element always takes up the full width available (stretches out to the left and right as far as it can).

Two commonly used block elements are: `<p>` and `<div>`.

- ❑ The `<p>` element defines a paragraph in an HTML document.
- ❑ The `<div>` element defines a division or a section in an HTML document.



Working with block elements and objects

Block-level Elements

- A block-level element always starts on a new line, and the **browsers automatically add some space (a margin) before and after the element.**
- A block-level element always takes up the full width available (stretches out to the left and right as far as it can).

```
<p style="border: 1px solid black">This is how block elements works. </p>
```

This is how block elements works.



Working with block elements and objects

Block-level Elements

```
<!DOCTYPE html>
<html>
<body>

<p style="border: 1px solid green">Hello World</p>
<div style="border: 1px solid red">Hello World</div>
```

<p>The P and the DIV elements are both block elements, and they will always start on a new line and take up the full width available (stretches out to the left and right as far as it can).</p>

```
</body>
</html>
```

Hello World

Hello World

The P and the DIV elements are both block elements, and they will always start on a new line and take up the full width available (stretches out to the left and right as far as it can).



Working with block elements and objects

Block-level Elements

HTML Block Elements

<address>	<article>	<aside>	<blockquote>	<canvas>
<dd>	<div>	<dl>	<dt>	<fieldset>
<figcaption>	<figure>	<footer>	<form>	<h1> - <h6>
<header>	<hr>		<main>	<nav>
<noscript>		<p>	<pre>	<section>
<table>	<tfoot>		<video>	



Working with block elements and objects

The <div> Element

- The <div> element is often used as a container for other HTML elements.
- The <div> element has no required attributes, but style, class and id are common.

When used together with CSS, the <div> element can be used to style blocks of

```
<!DOCTYPE html>
<html>
<head>
<style>
.myDiv {
  border: 5px outset red;
  background-color: lightblue;
  text-align: center;
}
</style>
</head>
<body>

<h1>The div element</h1>

<div class="myDiv">
  <h2>This is a heading in a div element</h2>
  <p>This is some text in a div element.</p>
</div>

<p>This is some text outside the div element.</p>

</body>
</html>
```

The div element

This is a heading in a div element

This is some text in a div element.

This is some text outside the div element.



Working with block elements and objects

HTML Inline Elements

Inline elements can appear within the same line and do not start a new line on their own.

Inline elements are displayed on the same line. They do not start on a new line and take up only as much width as their contents require. An example of an inline element is the **** tag.

```
<p>This is how <span style="border: 1px solid black">span</span> works. </p>
```

This is how span works.



Working with block elements and objects

HTML Inline Elements

If we have multiple span tags, they get placed on the same line.

For example,

<p>The following spans will be inline; Span 1 and Span 2.</p>

The following spans will be inline; Span 1 and Span 2.



Working with block elements and objects

HTML Inline Elements

Inline elements can appear within the same line and do not start a new line on their own.

List of HTML Inline Elements

The following table lists down all the inline elements –

HTML Inline Elements				
<a>	<abbr>	<acronym>		<bdo>
<big>	 	<button>	<cite>	<code>
<dfn>		<i>		<input>
<kbd>	<label>	<map>	<object>	<output>
<q>	<samp>	<script>	<select>	<small>
		<sub >	<sup>	<textarea>
<time>	<tt>	<var>		



Working with block elements and objects

HTML Inline Elements

EXAMPLE:

```
<!DOCTYPE html>
<html>

<head>
  <title>HTML inline Element</title>
</head>

<body>
  <h3>Inline Elements in HTML</h3>
  <!-- Using <b> inline element -->
  <p>This <b>paragraph</b> is bold. </p>
  <!-- Using <i> inline element -->
  <p>This is an <i>italic</i> paragraph.</p>
</body>

</html>
```

Inline Elements in HTML

This paragraph is bold.

This is an *italic* paragraph.



Working with block elements and objects

Grouping Block and Inline Elements

Block-level and inline elements can be grouped using the `<div>` tag. The `<div>` tag is a block-level element that plays a big role in grouping various other HTML tags and applying CSS to groups of elements.

Example:

This example demonstrates the group of elements using the `div` tag –



Working with block elements and objects

Grouping Block and Inline Elements

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML div Tag</title>
</head>
<body>
  <!-- First group of tags -->
  <div style="background-color:yellow">
    <h4>This is first group</h4>
    <p>Following is a list of vegetables</p>
    <ul>
      <li>Beetroot</li>
      <li>Ginger</li>
      <li>Potato</li>
      <li>Radish</li>
    </ul>
```

```
</div>
  <!-- Second group of tags -->
  <div style="background-color:cyan">
    <h4>This is second group</h4>
    <p>Following is a list of fruits</p>
    <ul>
      <li>Apple</li>
      <li>Banana</li>
      <li>Mango</li>
      <li>Strawberry</li>
    </ul>
  </div>
</body>
</html>
```



Working with block elements and objects

Grouping Block and Inline Elements

Output:

This is first group

Following is a list of vegetables

- Beetroot
- Ginger
- Potato
- Radish

This is second group

Following is a list of fruits

- Apple
- Banana
- Mango
- Strawberry



CSS Id and Class

CSS #id Selector

The ID selector in CSS is used to select a single element on a page by referencing its id attribute. This attribute must be unique within a page, meaning no two elements can have the same id. The ID selector is prefixed with a hash (#) symbol in CSS.

They serve different purposes and have distinct characteristics:

- **Uniqueness:** An id attribute in HTML must be **unique within a single HTML document**. This means **only one** element on a page can have a specific id.
- **Purpose:** Primarily used for uniquely identifying a single element, such as a main header, navigation bar, or a specific section that requires distinct styling or JavaScript interaction.



CSS Id and Class

Basic ID Selector

The ID selector allows you to style a specific element by targeting its unique id attribute. This is perfect for applying custom styles to an element that only appears once on the page.

```
<html>
<head>
  <style>
    #header {
      color: blue;
      font-size: 24px;
      text-align: center;
    }
  </style>
</head>
<body>
  <div id="header">Header Content</div>
</body>
</html>
```

Header Content

Syntax

```
#elementId {
  /* styles */
}
```

Where element Id is the value of the id attribute on the HTML element you want to target.



CSS Id and Class

Class Selector (.)

- **Multiplicity:** A class attribute can be applied to **multiple HTML elements, and multiple classes can be applied to a single HTML element.**
- **Purpose:** Used **for applying common styles to a group of elements that share a similar appearance or function.** This promotes reusability and maintainable code.



CSS Id and Class

CSS CLASS

- The CSS *.class* selector **selects elements with a specific class attribute value.**
- To select all kinds of elements with a specific class, write a period (.) character, followed by the class attribute value.
- The *element.class* selector selects the specified elements with the specified class attribute value.
- To select only one type of elements with a specific class, write the element name, then a period (.) character, followed by the class attribute value .

```
<!DOCTYPE html>
<html>
<head>
<style>
.intro {
    background-color: yellow;
}
</style>
</head>
<body>

<h1>Demo of the .class selector</h1>

<div class="intro">
    <p>My name is Donald.</p>
    <p>I live in Duckburg.</p>
</div>

<p>My best friend is Mickey.</p>

<p class="intro">My best friend is Mickey.</p>

</body>
</html>
```



CSS Id and Class

Key Differences Summarized:

Aspect	ID Selector (#)	Class Selector (.)
Uniqueness	Unique within the HTML document	Can be applied to multiple elements
Multiplicity	An element can have only one <code>id</code>	An element can have multiple <code>class</code> es
Usage	Specific, unique elements	Groups of elements with shared styles
Syntax	<code>#id_name</code>	<code>.class_name</code>



Box Model (Introduction)

The CSS Box Model is a fundamental concept in CSS that describes how every HTML element is rendered as a rectangular box on a web page. This model dictates how the dimensions and spacing of elements are calculated and presented in the browser.

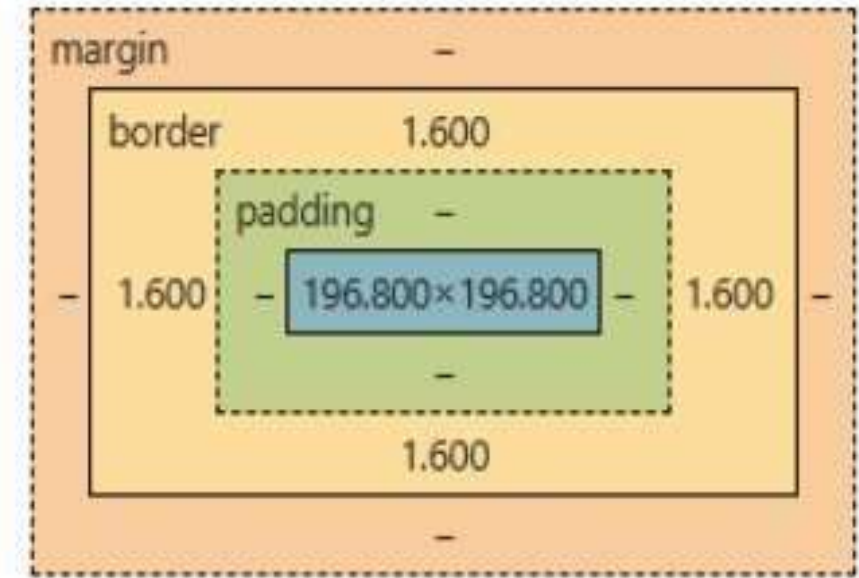
The CSS Box Model defines how elements are sized, positioned, and rendered on a webpage. When a browser loads an HTML document, it creates a [DOM tree](#) and assigns a box to each element. This box calculates the [element's dimensions](#) and [position relative](#) to its parent or the root <html> element, ensuring accurate layout and spacing.



Box Model (Introduction)

Box Model Component Layout

- **Content**: The area where text or other content is displayed.
- **Padding**: Space between the content and the element's border.
- **Border**: A frame that wraps around the padding and content.
- **Margin**: Space between the element's border and neighbouring elements.





Box Model (Introduction)

➤ CONTENT AREA:

- ❑ This is the innermost area where the actual content of the element resides, such as text, images, or other HTML elements. Its dimensions are typically controlled by the width and height CSS properties.
 - ❑ The content area is the central part of the CSS box model, containing the main content (e.g., text, images, videos, or elements like `<p>` or `<span>`).
- It can be styled with CSS properties like height and width.

The content edge refers to the four edges of the content area

- **Left content edge**
- **Right content edge**
- **Top content edge**
- **Bottom content edge**



Box Model (Introduction)

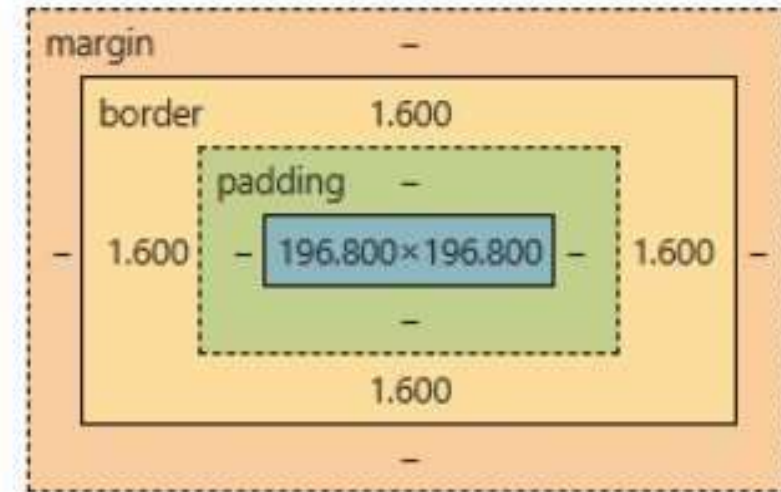
Each box is composed of four distinct areas, or layers, from the innermost to the outermost:

➤ CONTENT:

This is the innermost area where the actual content of the element resides, such as text, images, or other HTML elements. Its dimensions are typically controlled by the width and height CSS properties.

➤ PADDING:

This transparent space surrounds the content area, acting as a buffer between the content and the element's border. Padding is controlled by the padding property and takes on the background color of the element.





Box Model (Introduction)

➤ PADDING:

This transparent space surrounds the content area, acting as a buffer between the content and the element's border. Padding is controlled by the padding property and takes on the background color of the element.

- The padding area is the space between the content and the border of an element.
- It includes the areas highlighted in light green and skin color in the example.
- The distance between the [content edge](#) and the border is the padding.
- The border marks the end of the padding area.
- The padding area contributes to the element's total dimensions.
- Padding can be adjusted using [CSS properties](#).
- It works similarly with box-sizing: content-box and box-sizing: border-box, but with slight calculation differences.



Box Model (Introduction)

➤ **Border:**

- This line encloses the content and padding areas, providing a visual boundary for the element. The **border** property controls its style, width, and color.
- The area that marks the end of an element is called as the border it is the outer fencing for the element.
- The default border properties are provided in CSS to control the thickness of this outer fencing.
- The border area also add 's up to the complete height and width of the element.
- The more the border width the more will be the height or width of the element.
- In the above image the area marked with skin color is called the border area.



Box Model (Introduction)

➤ Margin:

This transparent space surrounds the border, creating a separation between the element and other adjacent elements on the page. Margins are controlled by the **margin** property and are always transparent, allowing the background of the parent element to show through.

- The area outside the border of an element is called the margin area.
- Basically this area depends on the parent of the element.
- The distance between the border of the parent element and the border of the child element is called as the margin.
- CSS has provides certain margin properties to get control over this scenario.



Box Model (Introduction)

The standard vs. alternative box model

Standard box model (`box-sizing: content-box`)

By default, CSS uses the standard box model, where the `width` and `height` properties apply only to the content area. [🔗](#)

- **Total width** = `width` + `padding-left` + `padding-right` + `border-left` + `border-right` + `margin-left` + `margin-right`.
- This can cause an element to be larger than its defined `width`, which can lead to unexpected layouts. [🔗](#)



Box Model (Introduction)

The standard vs. alternative box model

Alternative box model (`box-sizing: border-box`)

When `box-sizing` is set to `border-box`, the `width` and `height` properties include the content, padding, and border. 

- The content area shrinks to absorb any extra space from the padding and border.
- The total width and height of the element remain exactly as you specified, making sizing calculations much more predictable and easier to manage. 



Box Model(Border properties)

Box Sizing Property in CSS

1. Content-Box(default property)

When the user set's the value of the [box-sizing property](#) for an element as [content-box](#) or even if user do not set's it ,it remains by default as content-box and in the actual height and width of the element the dimensions of the content area as well as the padding area is added to constitute the final dimensions of the element.

```
<style>
  div {
    height: 200px;
    width: 200px;
    box-sizing: content-box;
    padding-left: 20px;
    padding-right: 20px;
    border-left: 2px solid red;
    border-right: 2px solid red;
  }
</style>
```



Box Model(Border properties)

Box Sizing Property in CSS

1. Content-Box(default property)

This code will create a box model with a [border line](#) width of 0.4px always and border-area of 1.6px and padding area as 20px width on both sides of the content area.

Content Area (Width):The width of the content area is fixed at 200px.

Padding

Padding adds extra space inside the element, around the content.

Padding Left: 20px

Padding Right: 20px

Total padding width: 20px + 20px = 40px

Border

The border, being solid, has a width, but it is calculated differently from the padding.

Line Width of Border: 0.4px (the width of the line itself)

Area of Border: 1.6px (the actual space the border occupies visually)

Border width for both sides: 1.6px (left) + 1.6px (right) = 3.2px



Box Model(Border properties)

Box Sizing Property in CSS

1. Content-Box(default property)

This code will create a box model with a [border line](#) width of 0.4px always and border-area of 1.6px and padding area as 20px width on both sides of the content area.

Total Width

Total width of the element can be calculated by adding the padding and border areas to the content area width.

Formula for Total Width = (Padding-Left + Padding-Right + Border-Area-Left + Border-Area-Right) + Content Area Width

Total Width = (20px + 20px + 1.6px + 1.6px) + 200px = 243.2px

The total width of the element becomes 243.2px.

The reason the total width is increased unexpectedly is because box-sizing: content-box applies the width to the content area only .The padding and border are added outside the content area, leading to an increase in the overall width and height of the element.



Box Model(Border properties)

Box Sizing Property in CSS

2. Border-Box:

When the box-sizing property is set as [border-box](#) the actual dimensions of the element's remains same as that of the actual dimensions set by the user. The difference it makes is just that the size of the content area get's altered in [a manner so that it could accommodate the padding area and the border area so the resultant could be equal to the actual dimensions entered by the](#)

user.

```
<style>
  div {
    height: 200px;
    width: 200px;
    box-sizing: border-box;
    padding-left: 20px;
    padding-right: 20px;
    border-left: 2px solid red;
    border-right: 2px solid red;
  }
</style>
```

This code will create a box model by altering the dimensions specifically the width of the content area to accomodate the padding and the border area with the border line-width.

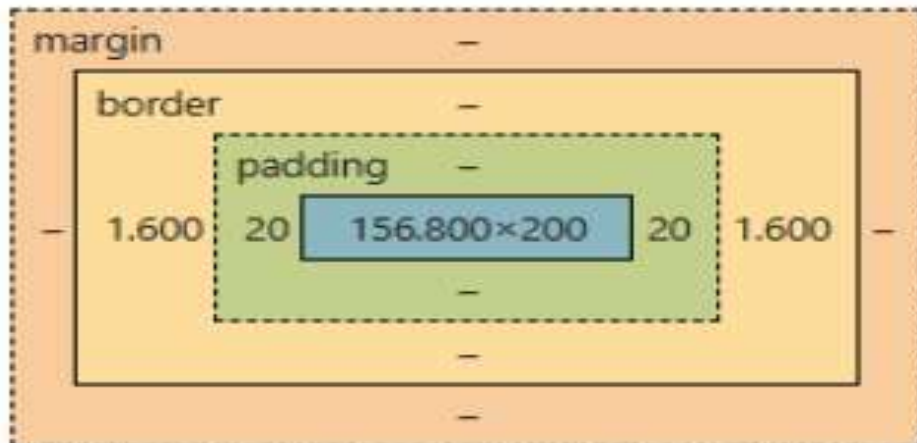


Box Model(Border properties)

Box Sizing Property in CSS

2. Border-Box:

```
<style>
  div {
    height: 200px;
    width: 200px;
    box-sizing: border-box;
    padding-left: 20px;
    padding-right: 20px;
    border-left: 2px solid red;
    border-right: 2px solid red;
  }
</style>
```



- **Width of Border and Padding** Border width : 0.4px (line width) and 1.6px + 1.6px = 3.2px (total border area).
- **Padding width** : 20px + 20px = 40px.
- **User-Entered Width** : The width entered by the user is 200px, which applies to the content area only when box-sizing: content-box is used.
- **Box-Sizing Behavior** :The box-sizing: content-box property adds the padding and border outside the content area, causing the total width to increase.
- **Adjusting Content Area Width** : To ensure the total width remains 200px, the extra width from padding and borders (40px + 3.2px = 43.2px) is subtracted from the total width.
- **New content area width** : 200px - 43.2px = 156.8px.
- **Final Width Calculation** : The final total width is: 156.8px (content area) + 40px (padding) + 3.2px (border) = 200px, ensuring the user's entered width remains unchanged.



CSS BORDERS

Borders in CSS are used to create a visible outline around an element.

They can be customized in terms of

- **Width:** The thickness of the border.
- **Style:** The appearance of the border (solid, dashed, dotted, etc.).
- **Color:** The color of the border.

You can try different types of borders here-

CSS Border Property

SolidDashed Red1rem SolidThick Double4mm Ridge

This is box

`border: 3px solid;`

Copy

Solid: A continuous line with solid borders.

Syntax:

```
element {  
    border: 1px solid black;  
}
```

*Here, the border is defined as **1 pixel wide, solid in style, and black in color.***



CSS BORDERS

```
<style>
  .simple-border {
    border: 2px solid black;
    padding: 20px;
    text-align: center;
  }
</style>
```

This code creates a div element with a simple black border around it, 2px wide. The content inside the div will have 20px of padding, which provides space between the border and the text. The text is centered using text-align: center.



CSS BORDERS

CSS Border Properties

CSS provides several properties to control and customize borders:

Property	Description
<code><u>border-style</u></code>	Determines the type of border (e.g., solid, dashed, dotted).
<code><u>border-width</u></code>	Sets the width of the border (in pixels, points, or other units).
<code><u>border-color</u></code>	Specifies the border color.
<code><u>border-radius</u></code>	Creates rounded corners for elements.



CSS BORDERS

Ways to Style Border in CSS

The CSS border property enables the styling of an element's border by setting its width, style, and color, allowing for customizable visual boundaries in web design.

1. Border Style

- [CSS border-top style Property](#)
- [border-right-style Property](#)
- [border-bottom-style Property](#)
- [border-left-style Property](#)

2. Border Width

- [border-top-width Property](#)
- [border-right-width Property](#)
- [border-bottom-width Property](#)
- [border-left-width Property](#)

3. Border Color

- [border-top-color Property](#)
- [border-right-color Property](#)
- [border-bottom-color Property](#)
- [border-left-color Property](#)

4. Border individual sides

CSS allows you to style each side of a border individually, giving flexibility in design.

5. Border Radius Property

The border-radius property allows you to round the corners of an element, creating smoother edges.



CSS BORDERS

1. Border Style

- CSS border-top style Property
- border-right-style Property
- border-bottom-style Property
- border-left-style Property

The **border-top style** property is used to specify the style of the top border.

Syntax:

```
border-top-style: none | dotted | dashed | solid | groove | inset |  
outset | ridge | double | hidden | initial | inherit;
```

Default Value: The default value is *none*.

The **border-right-style** property in CSS is used to change the appearance of the right line segment of the border of an element.

Default Value: none

Syntax:

```
border-right-style: none|hidden|dotted|dashed|solid|double|groove|  
ridge|inset|outset|inherit;
```



CSS BORDERS

```
<!DOCTYPE html>
<html>
  <head>
    <title>
      CSS | border-top-style Property
    </title>
    <style>
      h3.a { border-top-style: none;}
      h3.b {border-top-style: dotted; }
      h3.c {border-top-style: dashed; }
      h3.d {border-top-style: solid;}
      h3.e {border-top-style: groove;}
      h3.f {border-top-style: double;}
      h3.g {border-top-style: inset;}
      h3.h {border-top-style: outset; }
      h3.i {border-top-style: ridge;}
    </style>
  </head>
  <body>
    <h3 class="a">Rayat-Bahra Professional University </h3>
    <h3 class="b">Rayat-Bahra Professional University </h3>
    <h3 class="c">Rayat-Bahra Professional University </h3>
    <h3 class="d">Rayat-Bahra Professional University </h3>
    <h3 class="e">Rayat-Bahra Professional University </h3>
    <h3 class="f">Rayat-Bahra Professional University </h3>
    <h3 class="g">Rayat-Bahra Professional University </h3>
    <h3 class="h">Rayat-Bahra Professional University </h3>
    <h3 class="i">Rayat-Bahra Professional University </h3>
  </body>
</html>
```

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University

Rayat-Bahra Professional University



CSS BORDERS

Common Border Styles

The border-style property specifies the type of border. None of the other border properties will work without setting the border style.

Border Style	Description
Dotted	Creates a series of dots.
Dashed	Forms a dashed line.
Solid	Produces a continuous line.
Double	Renders two parallel lines.
Groove	Creates 3D grooved effect.
Ridge	Creates 3D ridged effect.
Inset	Adds 3D inset border.
Outset	Adds 3D outset border.
None	Removes the border.
Hidden	Hides the border.



CSS BORDERS

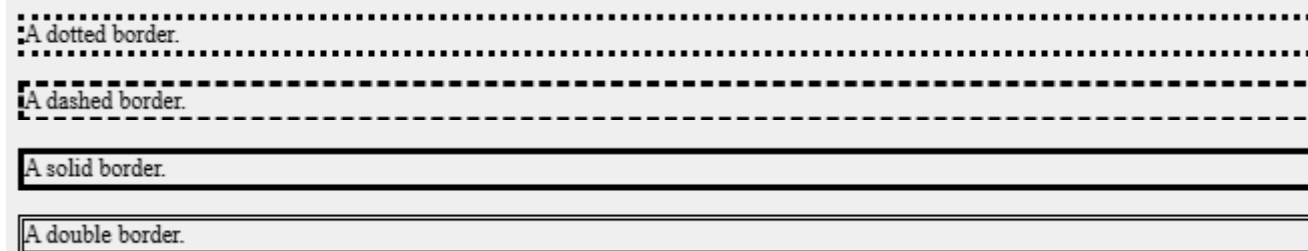
```
<html>
<head>

  <style>
    p.dotted {
      border-style: dotted;
    }
    p.dashed {
      border-style: dashed;
    }
    p.solid {
      border-style: solid;
    }
    p.double {
      border-style: double;
    }
  </style>

</head>
<body>
  <p class="dotted">A dotted border.</p>
  <p class="dashed">A dashed border.</p>
  <p class="solid">A solid border.</p>
  <p class="double">A double border.</p>
</body>
</html>
```

In this example:

- **border-style** is used to set the type of border around an element.
- **dotted**: Creates a border with dots.
- **dashed**: Creates a border with dashed lines.
- **solid**: Creates a solid, continuous border.
- **double**: Creates a border with two solid lines.





CSS BORDERS

CSS Border Width

CSS border-width is used to define the thickness of the border around an element. It can be specified in various units like px, pt, cm, or by using predefined values like thin, medium, and thick.

```
<html>
<head>

  <style>
    p {
      border-style: solid;
      border-width: 8px;
    }
  </style>

</head>
<body>
  <p>CSS Border Width</p>
</body>
</html>
```

In this example:

- Border-width property is used to set the thickness of the border.
- You can use numeric values (e.g., 1px, 5pt, 2cm) or keywords (thin, medium, thick) to set the border width.
- The border-style and border-color properties must be used in conjunction with border-width to see the effect.



CSS BORDERS

CSS Border Color

CSS border-color is used to define the color of the border. You can set the color using color names, hexadecimal values, or RGB values. If no color is specified, the border will inherit the color of the element itself.

```
<html>
<head>

  <style>
    p {
      border-style: solid;
      border-color: red
    }
  </style>

</head>
<body>
  <p>CSS Border color</p>
</body>
</html>
```

In this example:

- border-color property is used to set the color of the element's border.
- the border color is set to red using the color name red. You can also use hex codes like #ff0000 or RGB values like rgb(255, 0, 0).
- The border-style property must be defined (e.g., solid, dashed, etc.) for the border color to be visible.

CSS Border color



CSS BORDERS

Border radius property

The CSS border-radius property in CSS is used to round the corners of an element's border, giving it a more visually pleasing and smoother appearance.

```
<html>
<head>

  <style>
    h1 {
      border-style: solid;
      text-align: center;
      background: green;
      border-radius: 20px;
    }
  </style>

</head>
<body>
  <h1>Border Radius Property</h1>
</body>
</html>
```

In this example:

- border-radius is used to apply rounded corners to the element. In this case, a value of 20px is applied to create a soft, rounded edge.
- The element has a solid border, a green background, and centered text, with the rounded corners enhancing its visual appeal.
- You can adjust the border-radius value to control the curvature of the corners, making them more or less rounded.

Border Radius Property



Box Model(Padding Properties)

Padding is the **space between an element's content and its border**. It provides internal spacing inside the element, helping improve readability and the overall design of a web page.

Syntax:

```
selector{  
    padding: value;  
}
```

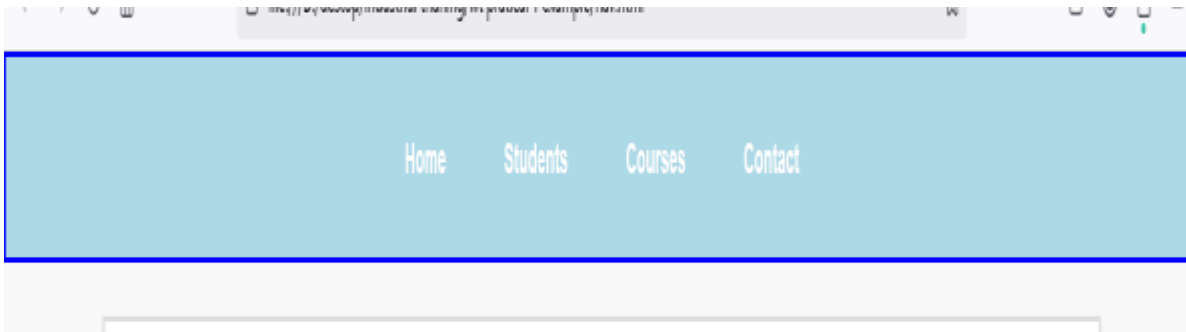


Box Model(Padding Properties)

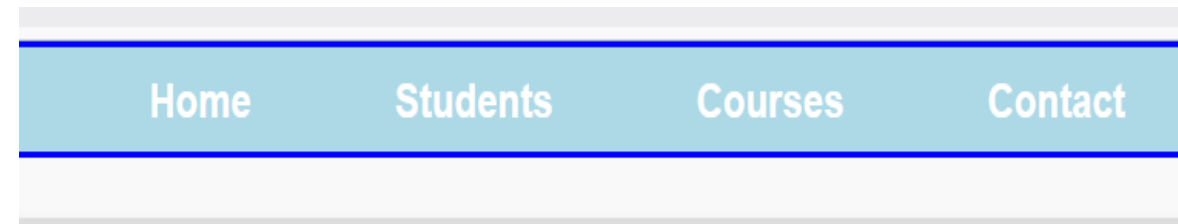
Method 1: Equal Padding on All Sides

When you specify a single value, it applies equally to top, right, bottom, and left.

```
/* Navigation Bar */
nav {
  background-color: lightblue;
  padding: 20px;
  Border: 2px solid blue;
}
```



```
/* Navigation Bar */
nav {
  background-color: lightblue;
  padding: 0;
  Border: 2px solid blue;
}
```





Box Model(Padding Properties)

Method 2: Vertical and Horizontal Padding When you provide two values, the first one sets the vertical padding (top and bottom), and the second sets the horizontal padding (left and right)

```
selector{  
    padding: value1 value2;  
}
```

```
/* Navigation Bar */  
nav {  
    background-color: lightblue;  
    padding:10px 100px;  
    Border: 2px solid blue;  
}
```



Box Model(Padding Properties)

Method 3: Individual Padding for Each Side When you provide four values, each corresponds to a different side:

```
selector{  
    padding: value1 value2 value3 value4;  
}
```

- Top: value1
- Right: value2
- Bottom: value3
- Left: value4



Box Model(Padding Properties)

Summary

- Single Value (padding: 20px) → Applies equal padding to all sides.
- Two Values (padding: 20px 50px) → Vertical | Horizontal.
- Four Values (padding: 15px 30px 25px 50px) → Top | Right | Bottom | Left.
- Padding helps give breathing space inside elements, making layouts look clean and user-friendly.



Box Model(Margin properties)

MARGIN PROPERTIES

- The margin is the outermost layer of the box model, creating space outside the border of an element, separating it from other elements on the page.
- Unlike padding, the margin area does not include the element's background color.

You can control the margin of an element using various CSS properties:

❑ **Shorthand margin property:** This allows you to set the margin for all four sides (top, right, bottom, left) in a single declaration.

One value: Applies to all four sides.

```
margin: 20px; /* 20px margin on all sides */
```



Box Model(Margin properties)

MARGIN PROPERTIES

Two values: The first value applies to the top and bottom, the second to the left and right.

```
margin: 10px 5px; /* 10px top/bottom, 5px left/right */
```

Three values: The first value applies to the top, the second to the left and right, and the third to the bottom.

```
margin: 10px 5px 15px; /* 10px top, 5px left/right, 15px bottom */
```



Box Model(Margin properties)

MARGIN PROPERTIES

Four values: Apply to top, right, bottom, and left respectively (clockwise).

```
margin: 10px 5px 15px 20px; /* 10px top, 5px right, 15px bottom, 20px left */
```

Individual margin properties: You can set the margin for each side independently.

margin-top: Sets the top margin.

margin-right: Sets the right margin.

margin-bottom: Sets the bottom margin.

margin-left: Sets the left margin.

```
margin-top: 10px;  
margin-right: 5px;  
margin-bottom: 15px;  
margin-left: 20px;
```



Box Model(Margin properties)

MARGIN PROPERTIES

All the margin properties can have the following values:

auto - the browser calculates the margin

length - specifies a margin in px, pt, cm, etc.

% - specifies a margin in % of the width of the containing element

inherit - specifies that the margin should be inherited from the parent element



Box Model(Margin properties)

MARGIN PROPERTIES

All the margin properties can have the following values:

auto - the browser calculates the margin

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 300px;
  margin: auto;
  border: 2px solid red;
}
</style>
</head>
<body>

<h2>Use of margin: auto</h2>
<p>margin property to auto to horizontally center </p>
<div>
This div will be horizontally centered because it has margin: auto;
</div>

</body>
</html>
```

Use of margin: auto

margin property to auto to horizontally center

This div will be horizontally centered because it has margin: auto;



Box Model(Margin properties)

MARGIN PROPERTIES

inherit - specifies that the margin should be inherited from the parent element

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  border: 1px solid red;
  margin-left: 100px;
}
```

```
p.ex1 {
  margin-left: inherit;
}
</style>
</head>
<body>
```

```
<h2>Use of the inherit value</h2>
```

```
<p>Let the left margin be inherited from the parent element:</p>
```

```
<div>
<p class="ex1">This paragraph has an inherited left margin (from the div element).</p>
</div>
```

```
</body>
```

```
</html>
```

Use of the inherit value

Let the left margin be inherited from the parent element:

This paragraph has an inherited left margin (from the div element).